

What is OOP?

PROGRAMMING PARADIGMS

HOW to do it
(step-by-step)

WHAT to do (let
(let system figure it out)

IMPERATIVE

DECLARATIVE



Procedural
Functions &
Procedures



Object-Oriented
Objects & State



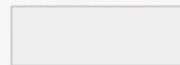
Functional
Pure Functions
& Data



Reactive



Logic



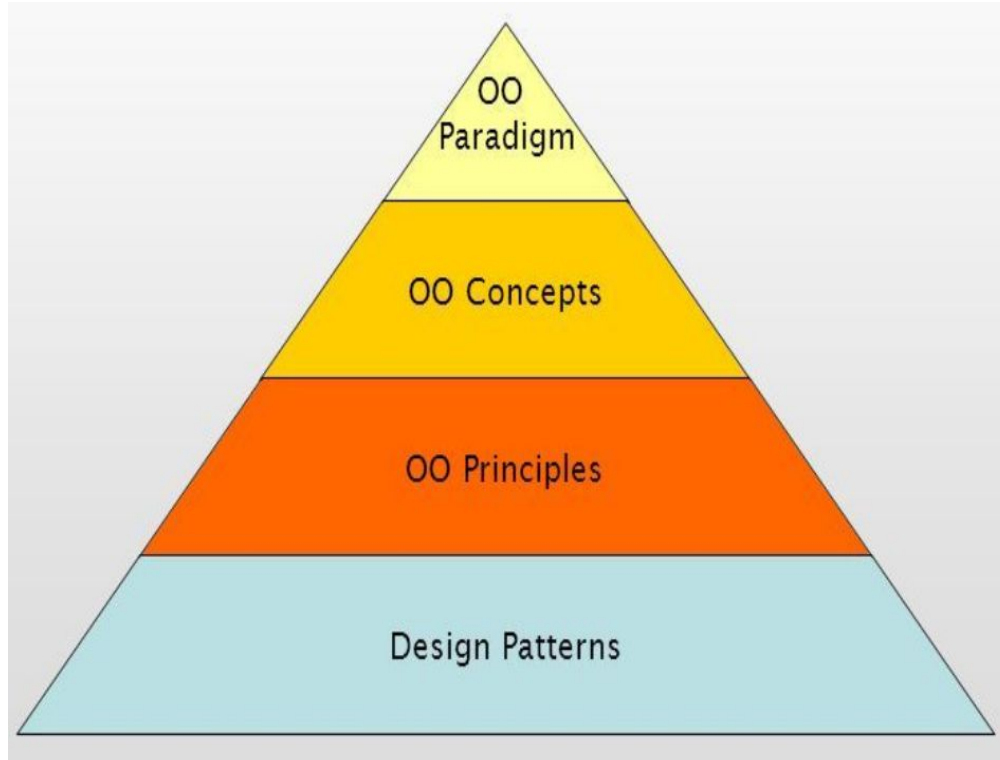
Imperative

```
Customer getById(int id){  
    for(auto c: customers){  
        if(c.id == id){  
            return c;  
        }  
    }  
    return Customer();  
}
```

Declarative

```
CREATE PROCEDURE GetById(IN custId INT)  
BEGIN  
    SELECT *  
    FROM customers  
    WHERE id = custId;  
END
```

Pyramid of OOP



OO Paradigm

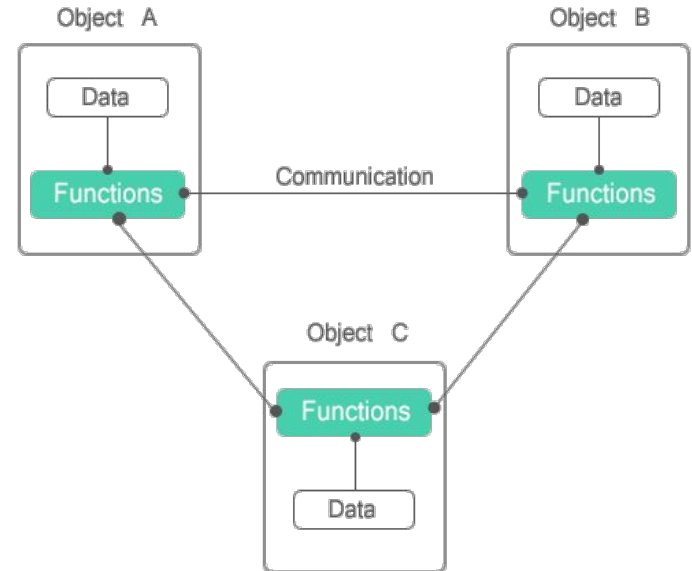
- organizes software design around **objects**
- **objects** with **behaviours** and **attributes** **interact** with each other in given **context**

- **WHY?**

- **Remember Procedural?**

- set of functions with linear flow
 - harder to debug
 - harder to modification and extension
 - no owner of data, data integrity issues, and more..

- OOP Classes Like Real World Object!
 - Help in Handling software **complexity**
 - Easier to thinking and debugging
 - **Reusability** → Inheritance
 - **Extensibility** → Polymorphism
 - **Data Hiding** → Encapsulation
 - **Complexity Hiding** → Abstraction



	Procedure Oriented Programming	Object Oriented Programming
Divided Into	In POP, program is divided into small parts called functions .	In OOP, program is divided into parts called objects .
Importance	In POP, Importance is not given to data but to functions as well as sequence of actions to be done.	In OOP, Importance is given to the data rather than procedures or functions because it works as a real world .
Approach	POP follows Top Down approach .	OOP follows Bottom Up approach .
Access Specifiers	POP does not have any access specifier.	OOP has access specifiers named Public, Private, Protected, etc.
Data Moving	In POP, Data can move freely from function to function in the system.	In OOP, objects can move and communicate with each other through member functions.
Expansion	To add new data and function in POP is not so easy.	OOP provides an easy way to add new data and function.
Data Access	In POP, Most function uses Global data for sharing that can be accessed freely from function to function in the system.	In OOP, data can not move easily from function to function, it can be kept public or private so we can control the access of data.
Data Hiding	POP does not have any proper way for hiding data so it is less secure .	OOP provides Data Hiding so provides more security .
Overloading	In POP, Overloading is not possible.	In OOP, overloading is possible in the form of Function Overloading and Operator Overloading.
Examples	Example of POP are : C, VB, FORTRAN, Pascal.	Example of OOP are : C++, JAVA, VB.NET, C#.NET.

OO Process

- **Object-Oriented Analysis (OOA)**

- Analysis product requirement and understanding problem domain
- Identify objects, their attributes, and interactions
- **Output**→ Analysis model (use cases, sequence & activity diagrams,..) describing what the system should do

- **Object-Oriented Design (OOD)**

- Designing the solution →very critical skill (needs experience and practice)
- Apply the OO Concepts
- Design Principles & Patterns well (depends on some criteria)
- **Output**→Design model like Class Diagram

- **Object-Oriented Programming (OOP)**

- Implementation of Designed Solution
- **Output**→Deployable Software Solution (Working System)

Requirements, Team Size & Experience, Project scale →Decide how the OO Process is **applied**

Class Diagram

- **UML (Unified Modeling Language) diagram**

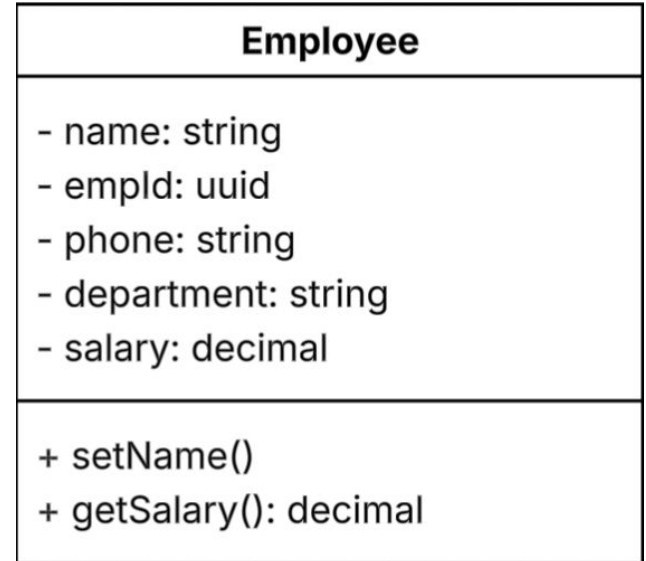
- Standard visualization of the system design before coding (or to document it)
- Defines how classes interact and depend on each other
- Used in OOD

- **Class Naming Convention**

- ClassName
- <<AbstractClassName>>

- **Access Modifiers**

- private
- + public
- # protected
- ~ package/default



Class Diagram: Relationships

- **Association**

- **uses-a** relationship (**uni** & **bidirectional**)
- **no ownership**
- person **reads** book, Teacher **knows** students, student **enrolls** in course

- **Aggregation**

- **whole-part** relationship where the part can exist independently of the whole
- shared ownership
- person **has** book, department **has** students

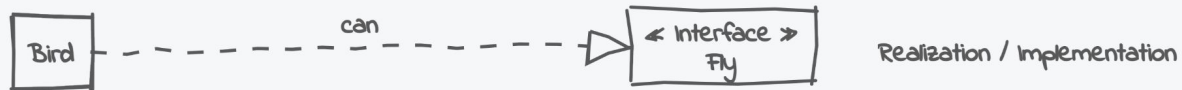
- **Composition**

- **strict whole-part** relationship where the part can exist independently of the whole
- exclusive ownership
- building **has** rooms/floors, university **has** faculties, **vehicle** **has** engine/wheels

- **Generalization**

- **is-a** relationship (class level relationship)
- Employee/Student is a Person, Car/Plane/Motorcycle is a Vehicle

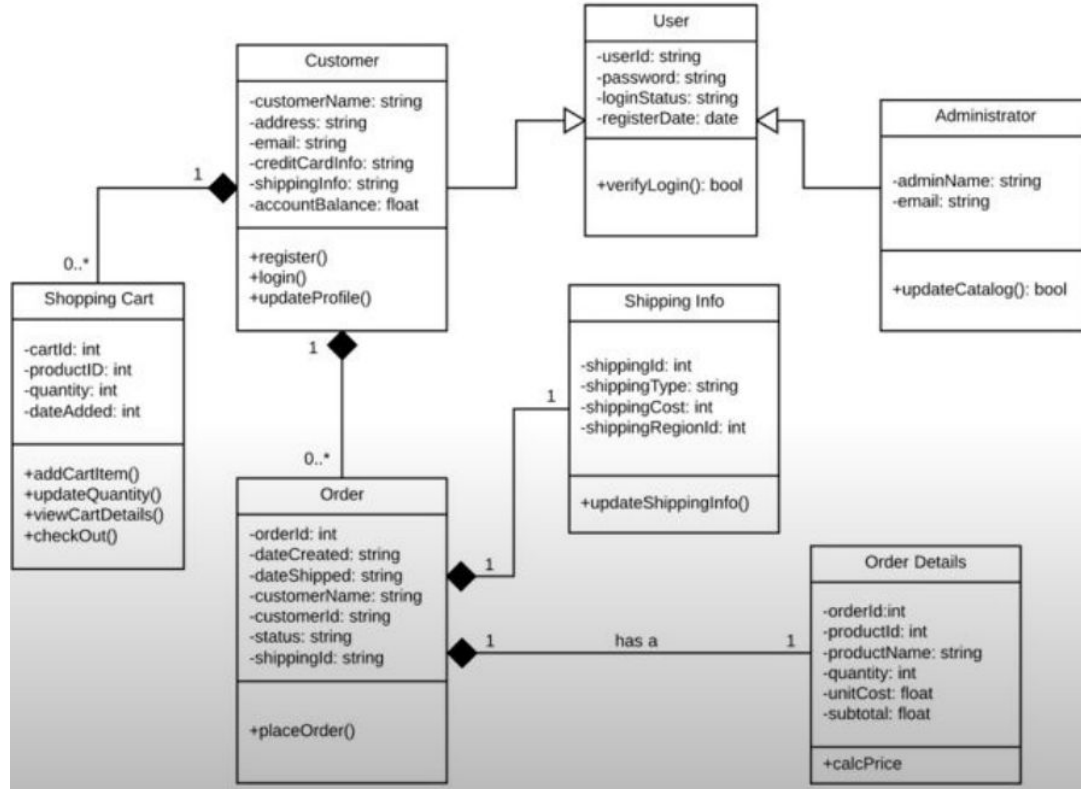
Class Diagram Notations

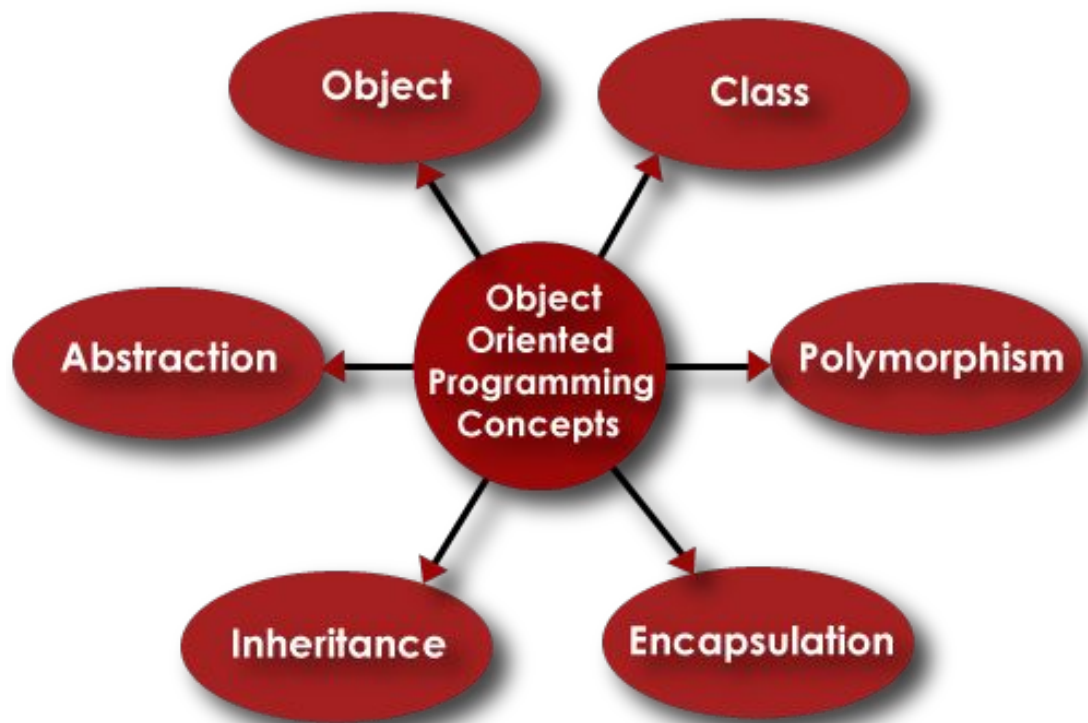


E-Commerce System UML: Hands On

An online store allows customers to register, browse products, add items to a shopping cart, and place orders

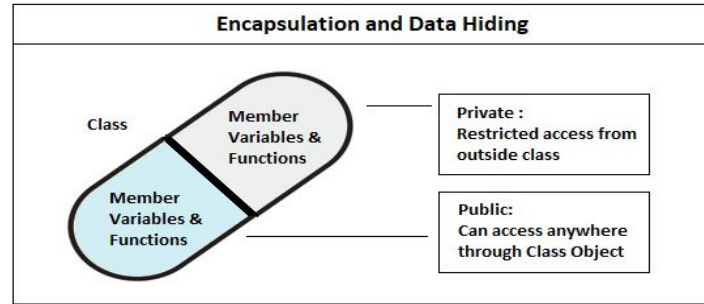
E-Commerce Class Diagram Solution





OOP Concepts: **Encapsulation**

- **Encapsulate** data and methods within single capsule
 - Administrate the Accessibility → **Access Modifiers**
 - Data Protection & Reduce the complexity → **Data Hiding**
- **Encapsulation** is a general concept of **grouping** related data and behavior into a **single controlled unit**
 - Data+Methods in a class (OOP)
 - Attributes in a table (Database)
 - Project in a single war file (Monolith) or multiple services (Microservices) arch.
 - Instance of running program in a Process (OS)



```
string per_name;  
int per_id;  
int per_age;  
string per_children_names[CHILDREN_COUNT];  
//..
```

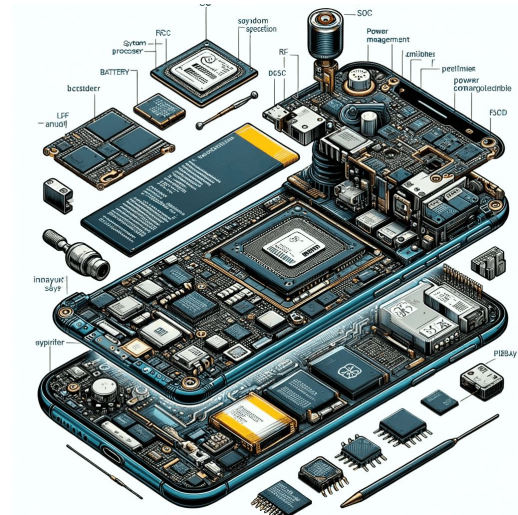
```
string per_name[MAX];  
int per_id[MAX];  
int per_age[MAX];  
string per_children_names[MAX][CHILDREN_COUNT];  
//..
```

```
class Person{  
    string name;  
    int id;  
    int age;  
    string children_names[CHILDREN_COUNT];  
    //..  
public:  
    //constructors, setters, getters  
};
```

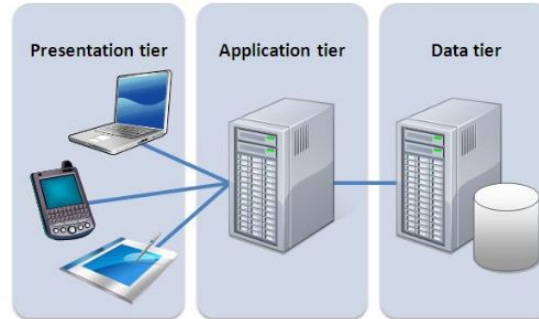
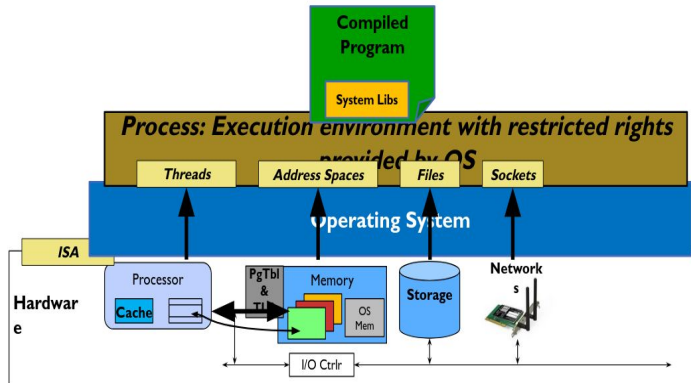
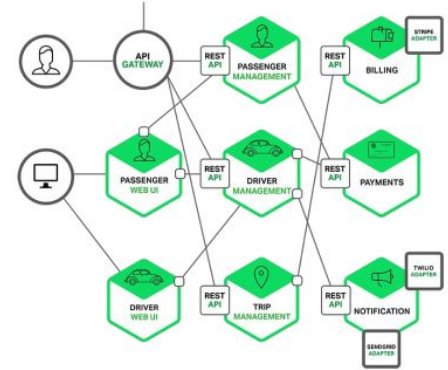
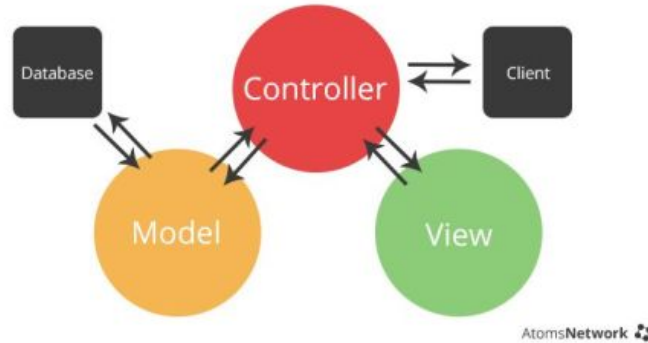
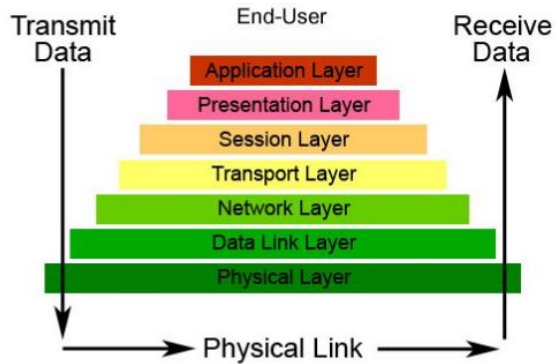
```
Person persons[MAX];
```

OOP Concepts: **Abstraction**

- **Hiding** unwanted details and show **only essentials**
 - User care about **What** (feature) not **How** (implementation)
 - Reducing system complexity



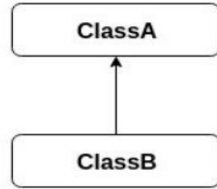
Encapsulation & Abstraction Examples



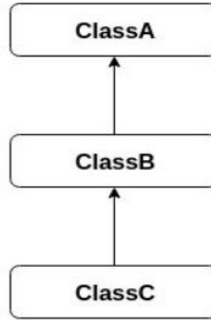
OOP Concepts: **Inheritance**

- **Subclass inherits** behavior and attributes of **superclass** (*is-a*)
 - Allows building hierarchy of classes and relations (**always advantage?**)
 - Allows **Reusability**
 - Enables **extensibility** and **polymorphism**
- **Tip: Don't** use it for **Reusability** Only!
 - Avoid Inheritance as much as possible
 - Favor Composition over Inheritance
 - Find another way using Interfaces + Composition + Polymorphism
 - more flexible and loosely coupled
 - Be aware of inheritance problems, seems works now, but future?
 - may be wrong, irrelevant, huge hierarchy, inflexible, buggy, confuses later!

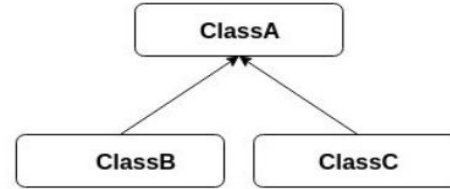
Inheritance Types



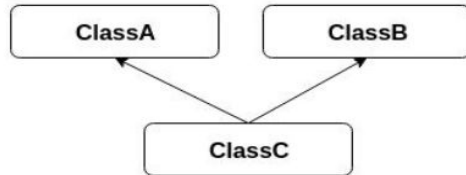
Single Inheritance



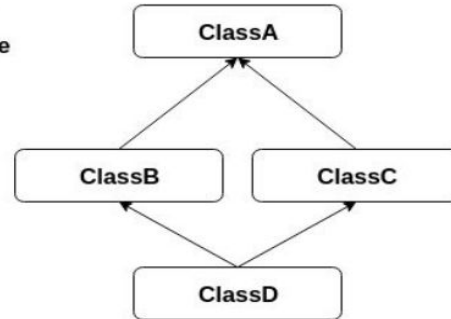
Multilevel Inheritance



Hierarchical Inheritance

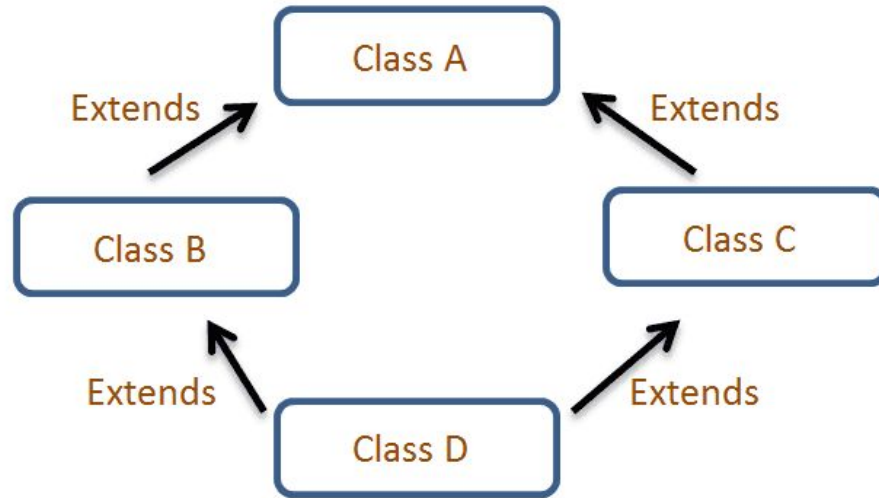


Multiple Inheritance



Hybrid Inheritance

Diamond Problem



OOP Concepts: **Polymorphism**

- **Many Forms** → one interface, many implementation

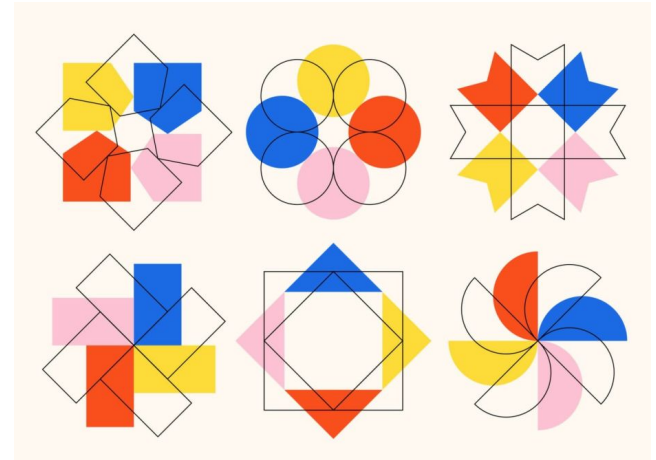
- Use it well → build **loosely coupled** system
- Allowing more generic code
 - Calculate the total invoice cost that contain set of **any** product
 - Calculate the area of **any** shape
 - More types can be added in future with minimal code changes

- **Static Polymorphism** (*Compile Time*)

- Function & Operator Overloading
- Without Inheritance

- **Dynamic Polymorphism** (*Run Time*)

- Function Overriding
- With Inheritance
- Upcasting: **generic code**
- Downcasting
 - if/else are unhealthy → use **OCP**



Interface Design Guidelines

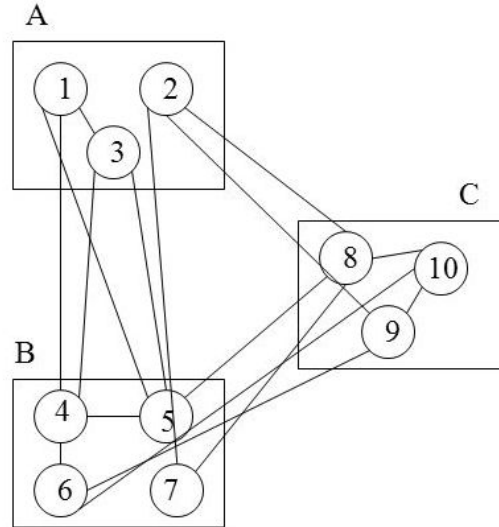
- Interface = **Contract** → **Don't Break It**
 - Any change can cause compatibility or compilation issues (**default** feature)
 - Design method names and parameters **carefully**
- Think from client/user perspective
 - **KISS** & make their life easy
- Keep It Minimal & Focused → *Interface Segregation Principle*
- Follow the Principle of Least Surprise (**POLA**)
 - Behavior should match expectations
 - Function names must reflect what they do

Interface (**can-do**) vs Abstract Class (**is-a**)

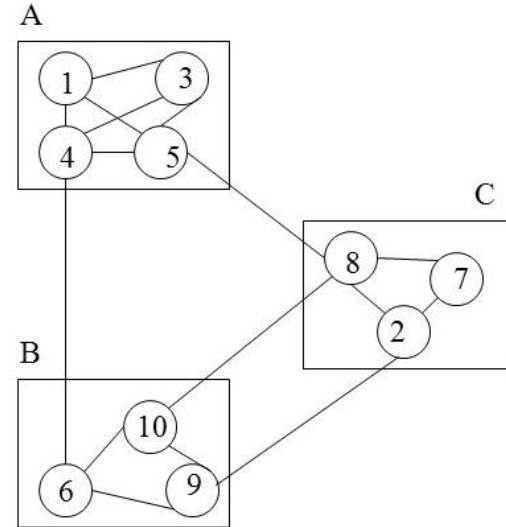
Feature	Interface	Abstract Class
Primary Purpose	Defines a contract (what an object can do).	Provides a base blueprint (what an object is).
Methods	Abstract by default (until Java 8). Can have <code>default</code> and <code>static</code> methods.	Can have both abstract and concrete methods.
Variables	Only <code>public static final</code> (constants).	Any type of variable (instance, static, etc.).
Inheritance	A class can implement multiple interfaces.	A class can extend only one abstract class (single inheritance).
Constructors	No constructors.	Yes , can have constructors (called when a subclass is instantiated).
When to Use	To define a common behavior for unrelated classes. To achieve polymorphism and loose coupling.	To share code and common state among closely related classes.

Coupling vs Cohesion

- **Coupling** is the degree of interdependence between modules/classes
- **Cohesion** is how closely the responsibilities within a single module/class are related



Bad modularization:
low cohesion, high coupling



Good modularization:
high cohesion, low coupling

Low Coupling & High Cohesion → Good Software Design